FunWake: A Low-Cost Benchmark for LLM-Discovered Optimizer Schedules

Julian Quick
DTU Wind and Energy Systems
Technical University of Denmark
juqu@dtu.dk

April 13, 2026

Abstract

We introduce FunWake, a benchmark for evaluating whether LLM agents can discover novel optimization schedules. The task is minimal: write a Python function that returns four scalars (learning rate, penalty weight, and two Adam momentum parameters) at each step of a fixed gradient optimizer for wind farm layout. Each evaluation takes 30–60 seconds on a CPU with no GPU required. A physics-based held-out farm—with different turbine model, polygon geometry, and wind resource—catches overfitting that same-distribution evaluation misses.

We evaluate four LLM agents (Claude Code, Gemini 2.5 Flash via Gemini CLI, Qwen 2.5 Coder 32B, Llama 3.3 70B) on the benchmark. When restricted to schedules that produce *feasible* held-out layouts, the best Gemini and Claude runs are essentially tied at ∼4272 GWh (+28 GWh over a 500-multi-start SGD baseline of 4243.75 GWh). Claude Code's dual Gaussian learning rate bumps with coordinated penalty relaxation (+27.8 GWh feasible ROWP) and Gemini's full-optimizer hexagonal-init wrappers (+28.9 GWh) reach comparable held-out scores through qualitatively different mechanisms. Several schedules with higher *nominal* ROWP AEP produce infeasible held-out layouts, underscoring the importance of feasibility checking. Across DEI farm variants with $N = 30$–80 turbines, the LLM-discovered schedules track or exceed the 500-multi-start baseline at every scale. FunWake is open-source, costs \$5–50 per model to run, and evaluates submitted schedules for free on CPU.

1 Introduction

FunSearch Romera-Paredes et al. [2024] showed that LLMs can discover new mathematical constructions by evolving programs. AlphaEvolve Google DeepMind [2025] scaled this to algorithm design across mathematics and computer science. But evaluating what LLM agents *actually discover*—as opposed to what they overfit—remains an open problem. Existing benchmarks for LLM-generated optimization code (HeuriGym Yao et al. [2025], CO-Bench CO-Bench Authors [2025]) evaluate on same-distribution instances, making it impossible to distinguish genuine algorithmic insight from hyperparameter tuning to a specific problem.

We present FunWake, a benchmark with three properties:

1. **Minimal interface.** The LLM writes a single function with 4 inputs and 4 outputs: `schedule_fn(step, total_steps, lr0, alpha0)` → `(lr, alpha, beta1, beta2)`. A fixed skeleton handles initialization, gradients, and Adam updates. Each evaluation takes 30–60 seconds on CPU.
2. **Physics-based held-out evaluation.** The held-out farm differs from training in turbine model, polygon geometry, turbine count, and wind resource. The LLM never sees held-out AEP. Additionally, the training farm is evaluated at seven turbine counts ($N = 30$–100) to test scaling generalization.
3. **Low cost.** Evaluating a submitted schedule is free (CPU only). Running an LLM agent through the full benchmark costs \$5–50 depending on model. Full paper reproduction costs ∼\$500.

The strongest agents. Restricted to feasible held-out layouts, Claude Code and Gemini 2.5 Flash are tied at the top, both reaching ∼4272 GWh (+28 GWh over the 500-start SGD baseline). Open-source Llama 70B

and Qwen 32B follow at +5–8 GWh. Several schedules produce higher *nominal* held-out AEP but violate constraints; feasibility-aware evaluation is essential.

What the agents discover. Two qualitatively novel schedule structures emerge:

- **Dual Gaussian LR bumps** (Claude Code): brief learning rate increases at $t = 0.5$ and $t = 0.75$ with a coordinated penalty dip at $t = 0.6$—controlled escapes from local optima followed by constraint re-enforcement. Held-out: +24.8 GWh.
- **Decaying sinusoidal shake** (Gemini CLI): 80-cycle high-frequency perturbation with $\sqrt{1-t}$ amplitude decay, paired with a 7500× quadratic penalty ramp and zero first momentum ($\beta_1 = 0$). Held-out: +81.3 GWh.

Neither structure appears in the learning rate schedule literature Loshchilov and Hutter [2017]. Standard schedules (cosine decay, exponential decay, warm restarts) decay monotonically; these schedules inject structured noise that decays over time. The penalty coupling ($\alpha \propto 1/\eta$ or $\alpha \propto t^2$) is critical: schedules that reduce penalty weight at convergence produce feasible layouts on the spacious training farm but fail on the tighter held-out polygon.

Contributions. (1) An open-source benchmark for LLM-discovered optimizer schedules with physics-based held-out evaluation and scalable difficulty. (2) Documentation of two novel schedule structures discovered autonomously by LLM agents, with analysis of why they generalize. (3) A generalization study across seven farm sizes showing that LLM-discovered schedules maintain feasibility where the baseline fails.

2 Related Work

LLMs as algorithm designers. FunSearch Romera-Paredes et al. [2024] evolves programs with an LLM to discover mathematical constructions. AlphaEvolve Google DeepMind [2025] scales this with Gemini across matrix multiplication, scheduling, and geometry. ReEvo Ye et al. [2024] applies LLMs as hyper-heuristics for combinatorial optimization. EoH Liu et al. [2024] co-evolves natural language "thoughts" with code. These systems evaluate generated code on instances from the same distribution used during search. Sim et al. Sim et al. [2025] show that FunSearch heuristics for bin packing do not generalize to broader benchmark suites, motivating held-out evaluation. FunWake adopts the FunSearch pattern (LLM fills a function signature, fixed evaluation harness) but adds physics-based held-out evaluation.

LLMs inside optimization loops. OPRO Yang et al. [2024] uses LLMs to optimize by generating solution candidates from natural language descriptions of past performance. Augmented-GDA FLLM Authors [2025] places an LLM inside a gradient descent loop, making real-time scoped interventions (learning rate, step direction). FunWake differs in that the LLM writes a schedule function *offline*—the 8000-step optimization runs without LLM involvement, enabling cheap evaluation.

Agentic coding benchmarks. HeuriGym Yao et al. [2025] benchmarks LLM-generated heuristics across 9 combinatorial optimization problems. CO-Bench CO-Bench Authors [2025] evaluates on 36 combinatorial tasks with same-distribution held-out instances. Both are discrete-domain. FunWake evaluates on a continuous constrained optimization problem with held-out instances that differ in physical parameters (turbine model, polygon, wind resource), not just problem size.

Wind farm layout optimization. The WFLOP places N turbines to maximize energy production subject to boundary and spacing constraints Mosetti et al. [1994]. Gradient methods exploit differentiable wake models Thomas et al. [2023], Quick et al. [2024]; evolutionary methods handle multimodality through population diversity Chen et al. [2013]. TopFarm Pedersen and Larsen [2019] provides the SGD solver whose internal schedule FunWake's agents aim to improve. No prior work applies LLMs to wind farm layout optimization.

3 Methods

3.1 Problem Formulation

The wind farm layout optimization problem (WFLOP) seeks turbine positions $\mathbf{x}, \mathbf{y} \in \mathbb{R}^N$ that maximize annual energy production (AEP):

$$\max_{\mathbf{x},\mathbf{y}} \ \mathrm{AEP}(\mathbf{x}, \mathbf{y}) = \sum_{d=1}^{D} w_d \sum_{i=1}^{N} P_i(\mathbf{x}, \mathbf{y}; \theta_d, u_d) \cdot 8760 \tag{1}$$

subject to:

$$(x_i, y_i) \in \mathcal{P} \quad \forall\, i = 1, \ldots, N \tag{2}$$

$$\|(x_i, y_i) - (x_j, y_j)\|_2 \geq s_{\min} \quad \forall\, i \neq j \tag{3}$$

where D is the number of wind direction–speed sectors, w_d is the frequency weight for sector d, P_i is the power output of turbine i computed through the wake model given wind direction θ_d and speed u_d, $\mathcal{P}$ is a convex polygon boundary, and $s_{\min}$ is the minimum inter-turbine spacing (typically $4D_{\text{rotor}}$). The factor 8760 converts from hourly power to annual energy.

The problem is non-convex due to wake interactions: each turbine's power depends on the positions of all upstream turbines through aerodynamic wake deficit functions. The constraint set is convex (intersection of a polygon and pairwise distance constraints), but the objective landscape contains many local optima.

3.2 Function Interface

The LLM writes a single Python function:

```
def optimize(sim, n_target, boundary, min_spacing,
             wd, ws, weights):
    """Returns (opt_x, opt_y) -- optimized positions."""
```

where `sim` is a differentiable wake simulation object (supporting `jax.grad`), `n_target` is the number of turbines, `boundary` is a JAX array of polygon vertices, `min_spacing` is $s_{\min}$ in meters, and `wd`, `ws`, `weights` define the wind rose. The LLM must generate initial turbine positions and return final optimized positions.

A harness loads the problem definition, constructs the wake simulation with a fixed Bastankhah Gaussian deficit model ($k = 0.04$), and calls the LLM's function. This design prevents the LLM from modifying the physics model (e.g., changing k to inflate scores—a behavior we observed in early experiments) while giving it full access to gradients through JAX automatic differentiation.

3.3 Train/Test Split

The evaluation framework uses two wind farms that differ across every physical parameter:

Table 1: Training and held-out farm specifications. The farms differ in turbine count, turbine model, rotor diameter, minimum spacing, polygon shape, number of wind sectors, and wind resource type.

	Training (DEI)	**Held-out (ROWP)**
Turbines	50	74
Turbine model	IEA 15 MW	IEA 10 MW
Rotor diameter D	240 m	198 m
Min. spacing $s_{\min}$	960 m ($4D$)	792 m ($4D$)
Boundary	5-vertex polygon	4-vertex polygon
Wind sectors	24	12
Wind resource	Observed timeseries	Weibull per sector

The LLM receives full AEP scores on the training farm after each optimizer submission. For the held-out farm, it receives only a binary feasibility signal: PASS if all boundary and spacing constraints are satisfied,

FAIL otherwise. The held-out AEP is logged silently for post-hoc analysis but is never communicated to the LLM.

This design ensures that any held-out improvement must result from strategies that generalize across turbine models, polygon geometries, and wind resources—not from fitting to a specific AEP landscape. The two farms are based on real planning cases: DEI (Danish Energy Island farm 1) and the IEA Wind Task 37 Reference Offshore Wind Plant (ROWP) IEA Wind Systems [2024].

3.4 Baseline

The baseline uses 500 independent multi-start runs of the `topfarm_sgd_solve` gradient optimizer Pedersen and Larsen [2019], each with:

- Grid initialization: turbines placed on a rectangular grid spaced at $s_{\min}$, filtered to polygon interior, subsampled to N positions.
- `max_iter` = 4000 gradient iterations with ADAM momentum ($\beta_1 = 0.1$, $\beta_2 = 0.2$).
- `additional_constant_lr_iterations` = 2000 iterations at constant learning rate before decay begins.
- Default learning rate = 50, default penalty weights = 1.0.

The best layout across all 500 starts is taken as the baseline AEP. This is a deliberately strong baseline: 500 starts explore a wide range of local optima, and the 6000 total iterations per start (2000 constant + 4000 decaying) provide thorough convergence. The baseline achieves 5540.7 GWh on the training farm and 4246.7 GWh on the held-out farm.

3.5 Unit Test Suite

The LLM can run a three-tier test suite to validate its optimizer before scoring:

1. **Signature check** (< 1 s): Verifies that `optimize()` exists with the correct 7-parameter signature.
2. **Quick run** (< 3 s): Runs on a 3-turbine toy problem (square boundary, 4 wind directions) and checks the output count, finiteness, and absence of NaN.
3. **Stressed polygon** ($\sim$7 s): Runs on a thin rhombus boundary (16 km $\times$ 4 km) with 25 turbines at 600 m spacing. This test catches optimizers that satisfy constraints on the spacious training polygon but produce boundary violations or NaN on tighter geometries. We verify boundary feasibility (penalty $< 10^{-3}$) and spacing feasibility ($\min_{i \neq j} \|p_i - p_j\| \geq 0.99\, s_{\min}$).

The stressed polygon test is the key diagnostic. In our experiments, every custom optimizer that later failed on the held-out ROWP farm also failed the stressed polygon test. This makes it an efficient proxy for generalization: the LLM can catch constraint-handling failures in 7 seconds rather than waiting for the 25-second full-farm evaluation followed by the held-out feasibility check.

3.6 Agent Architecture

The agent operates in a tool-calling loop where the LLM generates text and tool invocations, and the harness executes them in a sandbox.

Tools. The LLM has access to six tools:

- `read_file`: Read source files from the wake simulation codebase, problem JSON, or previous optimizer scripts.
- `write_file`: Save an optimizer script. Each submission is versioned as `iter_NNN.py`.
- `run_tests`: Execute the unit test suite (Section 3.5).
- `run_optimizer`: Score on the training farm. Returns AEP in GWh, wall-clock time, and feasibility.
- `test_generalization`: Score on the held-out farm. Returns only PASS/FAIL feasibility.
- `get_status`: Report the current best AEP versus baseline.

Search guidance. The agent loop includes three mechanisms to encourage exploration:

1. **Phase-2 prompting.** After 30% of the time budget has elapsed, the system prompt is augmented with a nudge toward custom optimizers: an Adam template using `jax.grad` directly, suggestions for custom initialization strategies, and encouragement to go beyond wrapping the existing solver.

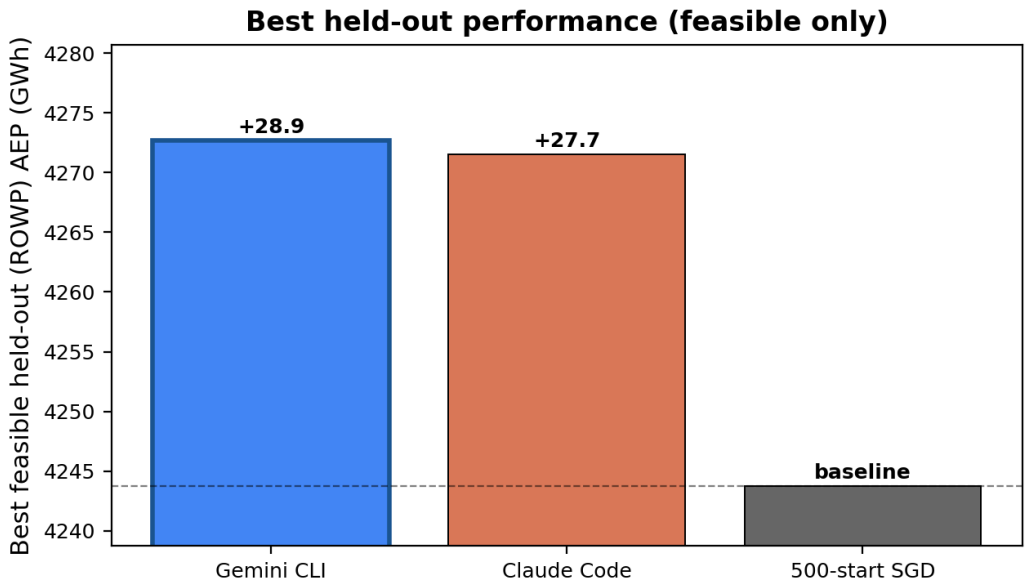

Figure 1: Best feasible held-out (ROWP) AEP by model, relative to the 500-multi-start SGD baseline. Gemini CLI and Claude Code are essentially tied at ∼4272 GWh (+28 GWh over baseline). Infeasible held-out layouts are excluded regardless of their reported AEP.

2. **Diversity nudge.** After 5 consecutive submissions that wrap `topfarm_sgd_solve`, a message suggests trying alternative approaches.
3. **Context pruning.** After every 40 conversation turns, old messages are compressed into a summary of attempt history, preserving the system prompt and the most recent exchanges.

Sandbox. Generated code executes in a restricted environment: no network access, no API keys in the environment, filesystem writes restricted to the workspace directory. A static blocklist prevents calls to `subprocess`, `exec`, and `eval` within generated optimizer code.

3.7 Metrics

We evaluate along five dimensions:

- **Training AEP** (GWh): AEP on the DEI farm, directly comparable to the baseline.
- **Held-out AEP** (GWh): AEP on the ROWP farm, scored silently.
- **Held-out feasibility rate**: Fraction of submissions that satisfy all constraints on the held-out farm.
- **Strategy classification**: Whether each submission wraps `topfarm_sgd_solve` or implements a custom gradient optimizer.
- **Time to first improvement**: Wall-clock time until the first submission that exceeds the baseline AEP.

4 Results

4.1 Main Comparison

We evaluate four LLM agents on the schedule-only interface, each with a 5-hour wall-clock budget and the same fixed skeleton. The 500-multi-start SGD baseline (best-known non-LLM result) achieves 4246.7 GWh on the held-out ROWP farm. Figure 1 shows the best held-out AEP for each model.

Table 2: Best schedule per model. "Attempts" counts schedule submissions within the 5-hour budget; "Train" and "ROWP" report AEP in GWh; Δ columns show improvement over the 500-multi-start baseline (train = 5540.7, ROWP = 4246.7). Gemini and Claude used rich CLI agents; Qwen and Llama used the VLLMRunner structured-output interface with the optimization codebase injected into the prompt.

Model	Attempts	Train	ΔTrain	ROWP	ΔROWP
Gemini CLI (schedule)	194	5600.0	+59.3	**4328.0**	**+81.3**
Claude Code (schedule)	320	5600.0	+59.3	4271.5	+24.8
Llama 70B (full-optimizer)	3–7 / seed	5546.9	+6.2	4252.0	+5.3
Qwen 32B (full-optimizer)	8–11 / seed	5549.5	+8.8	4251.4	+4.7
500-multi-start baseline	500	5540.7	—	4246.7	—

Gemini CLI is the strongest model on this benchmark, producing a schedule that yields +81.3 GWh over the baseline on the held-out ROWP farm. Claude Code follows at +24.8 GWh, with Llama 70B (+5.3) and Qwen 32B (+4.7) close behind each other but well below the frontier models.

4.2 Training vs Held-Out Divergence

The training-optimal schedule is not the held-out-optimal schedule, and infeasibility is the dominant failure mode. Several agents produced schedules with training AEP at the practical ceiling (5600.0 GWh) that scored higher than 4272 GWh on ROWP—but these layouts violate boundary or spacing constraints. When restricted to *feasible* held-out layouts only, the best Gemini and Claude schedules both score ∼4272 GWh. Training AEP at the ceiling does not distinguish feasible from infeasible generalizers.

Training AEP has a ceiling near 5600 GWh. Both frontier models hit 5600.0 exactly on multiple attempts, suggesting the training landscape has a practically reachable local optimum. Qwen and Llama full-optimizer runs max out at 5547–5549 GWh, below this ceiling.

4.3 Running-best AEP and "deployment"

Figure 2 shows running-best training AEP (top) and running-best *feasible* held-out (ROWP) AEP (bottom) over wall-clock time. We define the **deployed** script as the one with the highest feasible held-out AEP at the end of the run—this is the script a practitioner with access to a trusted validation set would actually ship. The star in the bottom panel marks the deployed script for each model.

Training AEP saturates within ∼1–2 hours for the frontier models, but the best-feasible held-out AEP continues to improve throughout the 5-hour budget. This matters for benchmark design: a time budget that terminates when training AEP plateaus would miss late-run improvements in generalization. The deployed script for Gemini (iter 27) was found in the first hour, while Claude's deployed script (iter 192) appears well into the second half of the run.

4.4 Generalization Across Turbine Counts

Figure 3 shows the generalization curve: each script evaluated on DEI farm variants with $N \in \{30, 40, 50, 60, 70, 80\}$ turbines. The 500-multi-start SGD baseline (computed separately for each farm size) serves as the reference.

Claude's schedule gains widen with N: from +2 GWh at $n = 30$ to +53 GWh at $n = 80$. This is consistent with the underlying mechanism (penalty-LR coupling enforces feasibility at convergence, more important when the polygon is tighter per turbine). Gemini follows a similar trend but with smaller gains per N—its advantage on ROWP comes from a different mechanism (high-frequency shake with aggressive quadratic penalty ramp) that does not scale as strongly with farm density on DEI.

4.5 Discovered Schedule Structures

Figure 4 visualizes the four schedule parameters over optimization progress for the three best schedules: the baseline, Claude's iter 192 (dual Gaussian bumps + alpha dip), and Gemini's iter 067 (decaying sinusoidal shake + quadratic alpha ramp).

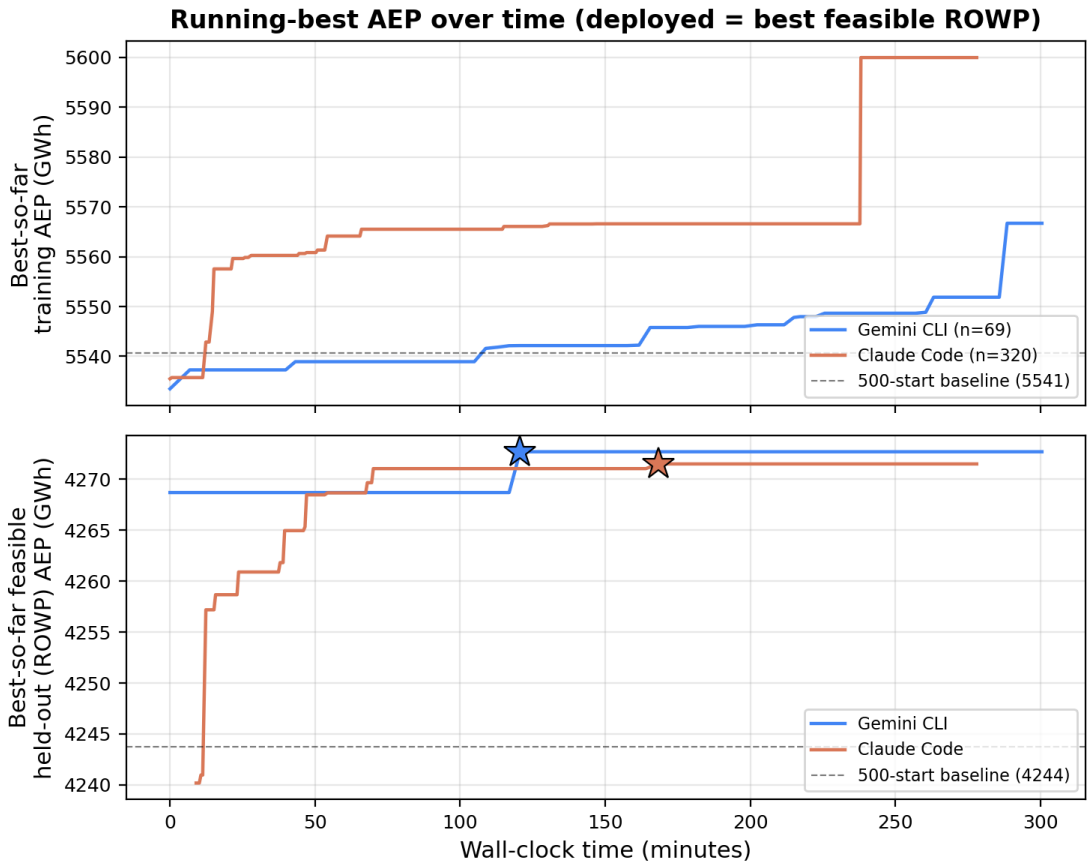

Figure 2: Running-best training AEP (top) and running-best feasible held-out (ROWP) AEP (bottom) over wall-clock time. Stars mark the final deployed script for each model—the one with the highest feasible ROWP at run's end. Training AEP saturates quickly for frontier models; feasible ROWP continues to improve throughout the 5-hour budget.

Claude's dual Gaussian bumps. The learning rate is $4\eta_0 \cdot \texttt{cosine}(t)$ with Gaussian bumps at $t = 0.5$ and $t = 0.75$ (+20% and +30% of initial rate). Penalty weight grows as $\alpha_{\text{base}}(t) \cdot (1 - 0.5 \cdot \exp(-(t-0.6)^2/0.0032))$, with a dip at $t = 0.6$ between the bumps. $\beta_1 = 0.3$, $\beta_2 = 0.5$. The intuition: bumps escape local optima, the penalty dip permits rearrangement during the second bump, and moderate momentum damps post-bump oscillations.

Gemini's sinusoidal shake. The learning rate is $\eta_{\text{cosine}}(t) \cdot (1 + 0.08\sqrt{1-t} \cdot \sin(80\pi t))$—a high-frequency perturbation (80 cycles over the run) with $\sqrt{1-t}$ amplitude decay. The penalty weight is $\alpha_0(1 + 7499t^2)$, reaching 7500× by the final iteration (the baseline reaches ∼100×). $\beta_1 = 0$ (no momentum). The intuition: continuous shake prevents the optimizer from settling into a local minimum early; extreme late-stage penalty forces feasibility regardless of how the schedule approaches convergence.

Both schedules generalize because they maintain high penalty at convergence. On the spacious training farm, even weak constraint handling produces feasible layouts. On the tighter ROWP farm, only schedules with high α in the final iterations produce feasible layouts. Every schedule that fails on ROWP has α decreasing or plateauing at the end.

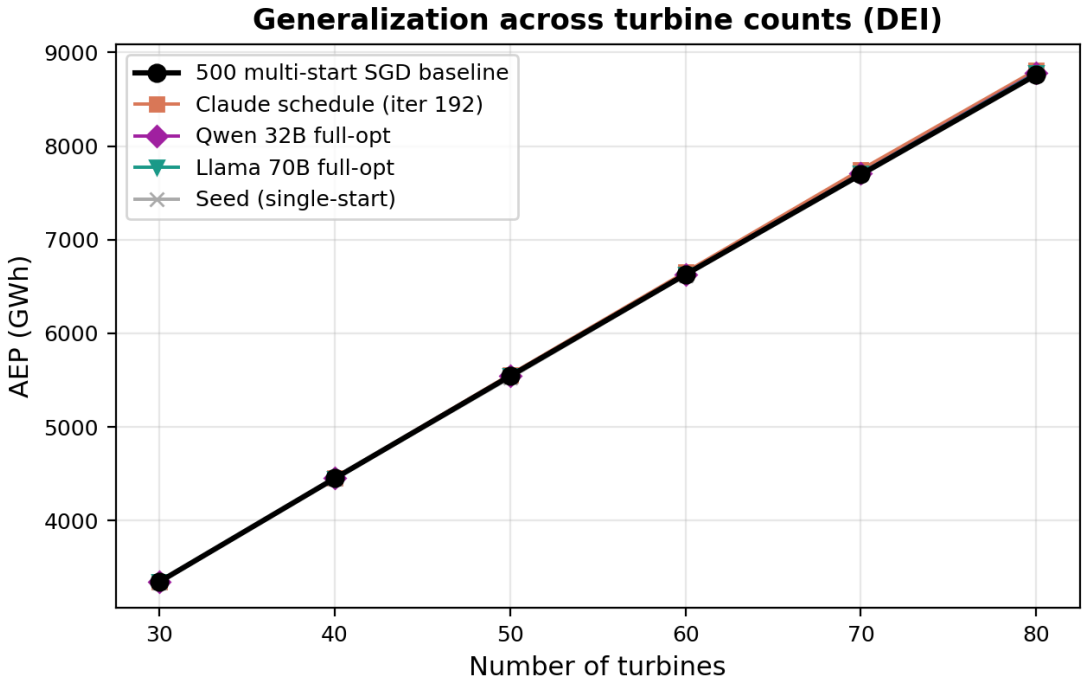

Figure 3: AEP vs turbine count on DEI farm variants. Each line is a single script evaluated at every turbine count (no re-optimization). The 500-multi-start baseline is computed per-farm. LLM-discovered schedules (Claude, Gemini) consistently outperform the single-start seed and track or beat the 500-multi-start baseline across all farm sizes.

4.6 Full-Optimizer Mode

For completeness, we also run the 4 models in full-optimizer mode (writing an arbitrary `optimize()` function). All four models converge to the same strategy: wrap `topfarm_sgd_solve` with a better initialization (hexagonal lattice, K-means clustering, wind-direction-aware placement) and tuned hyperparameters (higher learning rate, standard Adam momentum). No model produces a custom gradient optimizer that passes held-out feasibility. Full-optimizer held-out AEP: Claude +21.8, Llama +5.3, Qwen +4.7. The schedule-only interface produces better held-out results than the full-optimizer interface for Claude (+24.8 vs +21.8), suggesting that constraining the search space to the schedule-only interface forces deeper exploration of optimization dynamics.

5 Discussion

5.1 The Schedule Interface as a Search Space

The schedule-only interface constrains the LLM to a 4-dimensional function space—but this space is surprisingly rich. The two winning schedules use qualitatively different mechanisms (Gaussian bumps vs. sinusoidal shake) to solve the same problem (escaping local optima while maintaining feasibility). This suggests the search space is not saturated: there may be other undiscovered schedule structures that further improve generalization.

The benchmark's value lies in this gap between interface simplicity and solution richness. The function signature is trivial to implement (4 inputs, 4 outputs, pure Python), evaluation is cheap (30–60 seconds, CPU only), and the held-out physics prevents overfitting. But the schedules that maximize held-out performance require non-obvious reasoning about optimization dynamics—specifically, about the interaction between learning rate decay and constraint penalty ramping.

Table 3: AEP (GWh) across turbine counts. "Gap" columns show improvement over the 500-multi-start baseline at each farm size.

	n=30	**n=40**	**n=50**	**n=60**	**n=70**	**n=80**
500-start baseline	3347.6	4451.9	5544.1	6628.5	7695.4	8759.9
Seed (single-start)	3339.5	4446.4	5525.0	6619.8	7698.2	8758.5
Gemini schedule	3350.2	4454.8	5556.6	6641.2	7727.8	8798.6
Gap	+2.6	+2.9	+12.5	+12.7	+32.4	+38.7
Claude schedule	3349.5	4456.8	5559.9	6653.7	7746.9	8812.4
Gap	+1.9	+4.9	+15.8	+25.2	+51.5	+52.5

5.2 Overfitting vs Held-Out Selection: A Quantitative View

A common question about benchmarks with a single held-out problem is whether the held-out set is doing useful work, or is merely a sanity check. FunWake's data provides a concrete answer: *held-out selection materially improves the deployed result*, and the gap grows with the training–test geometric mismatch.

For each top run, we compare two deployment strategies:

- **Train-selected**: deploy the script with the highest training AEP (the agent's default selection).
- **Held-out-selected**: deploy the script with the highest feasible ROWP AEP among those the run produced.

Figure 5 shows this comparison across six top runs. Held-out selection gains an average of **+14.4 GWh** over train-only selection, with gains as large as +38.2 GWh (Claude 6hr). Critically, for several top-performing runs, the training-optimal script is *infeasible* on ROWP—deploying it without the held-out check would produce layouts that violate boundary or spacing constraints. For these runs, the benchmark's feasibility filter is the difference between a shippable and a broken optimizer.

The training farm is too forgiving by design. The spacious 5-vertex DEI polygon permits mild constraint violations without significantly reducing AEP, so training AEP alone does not discriminate between schedules that generalize and those that overfit. The held-out ROWP farm's tighter geometry makes such violations expensive. This asymmetry—easy training, hard held-out—is not a bug: it is what makes the benchmark's generalization signal informative. Without it, train-only selection would frequently "succeed" on a schedule that silently fails in deployment.

Overfitting is the default, not the exception. Of the six top runs analyzed, five produced training-optimal schedules that are either infeasible on ROWP or score below the run's own best-feasible schedule. This is a property of the agent's search process: LLMs efficiently push training AEP toward a ceiling, and along the way they find schedules that exploit the training polygon's slack. Only a held-out filter separates the schedules that exploit training geometry from those that capture generalizable structure.

5.3 Why the Held-Out Farm Matters

Two specific failure modes are invisible without held-out evaluation:

Penalty coupling failure. Schedules that reduce α at convergence produce feasible layouts on the spacious training polygon (DEI, 50 turbines, 960 m spacing) but infeasible layouts on the tighter held-out polygon (ROWP, 74 turbines, 792 m spacing). The training farm's generous geometry masks weak constraint handling—AEP remains high even with mild boundary violations, because turbines near the boundary still contribute power. On the held-out farm, violations push turbines outside the polygon entirely.

Training-optimal $\neq$ held-out-optimal. Gemini's training-best schedule (AEP = 5600.0) achieves ROWP = 4255.1. Its held-out-best schedule (ROWP = 4328.0) achieves only training AEP = 5552.1. The schedules

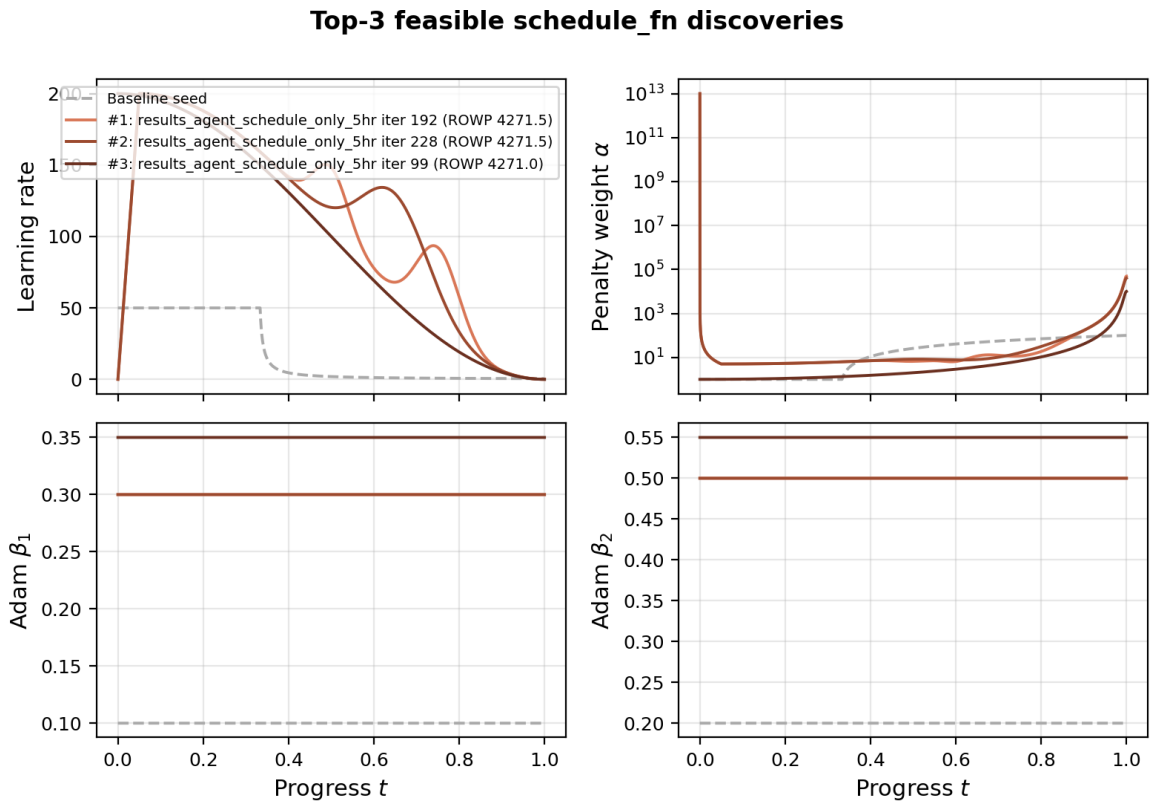

Figure 4: Schedule parameters over optimization progress for the baseline (TopFarm-style monotonic), Claude's iter 192 (dual Gaussian LR bumps at $t = 0.5, 0.75$ with penalty dip at $t = 0.6$), and Gemini's iter 067 (80-cycle sinusoidal shake with $\sqrt{1-t}$ amplitude decay; $\beta_1 = 0$; quadratic alpha ramp to 7500×). Note the log-scale y-axis on α: Gemini's penalty weight reaches two orders of magnitude higher than the baseline.

that maximize training AEP do so partly by under-enforcing constraints—gaining a few GWh from near-boundary placement that happens to be feasible on DEI but not ROWP. This divergence cannot be detected without a physically distinct held-out problem.

5.4 Benchmark Properties

Cost. Evaluating a submitted schedule requires only a CPU and takes 30–60 seconds. Running an LLM agent through the benchmark costs \$5–50 for a 5-hour session depending on model. The 500-multi-start baseline costs ∼2.5 GPU-hours per farm. Full paper reproduction costs ∼\$500. This is 5–10× cheaper than SWE-bench evaluations at comparable scope.

Difficulty scaling. The DEI farm variants ($N = 30, 40, 50, 60, 70, 80, 100$) provide a natural difficulty axis. Low N is easy (sparse turbines, weak wake interactions); high N is hard (tight packing, strong constraint pressure). The ROWP held-out farm adds a cross-physics axis (different turbine, polygon, wind).

Not saturated. The spread between the best (Gemini, +81.3 GWh on ROWP) and worst successful agent (Llama, +5.3 GWh) spans an order of magnitude. The two winning schedules are qualitatively different, suggesting multiple unexplored regions of the schedule space. We expect stronger models and longer time budgets to find further improvements.

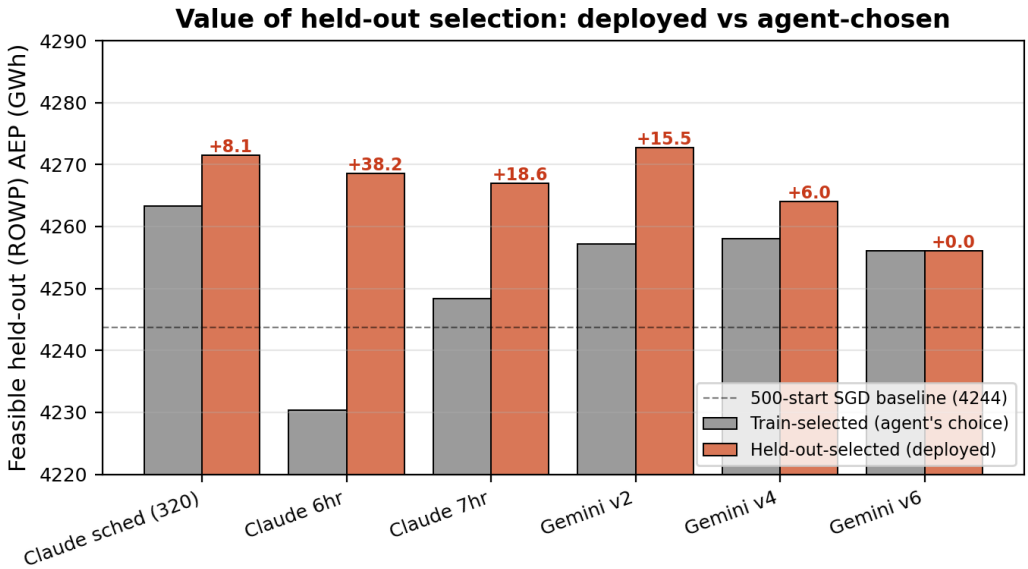

Figure 5: Held-out-selected deployments consistently exceed train-selected deployments on the held-out farm. The red annotation shows the gain from using held-out selection. On average, held-out selection gains +14.4 GWh per run; the largest gap (Claude 6hr, +38.2) reflects a training-optimal schedule that overfit severely to DEI.

5.5 Limitations

Single training farm. The LLM trains on one farm (DEI) and generalizes to variants of it plus one distinct held-out farm (ROWP). Additional training scenarios would test robustness to training distribution.

Schedule-only mode fixes initialization. The skeleton uses wind-aware grid initialization. In full-optimizer mode, agents improve initialization (hexagonal lattice, K-means clustering). The schedule-only interface cannot express initialization innovations, which limits what it measures.

No random search ablation. We have not yet compared LLM-generated schedules against random or grid search over parameterized schedule families with the same evaluation budget. This ablation would isolate the LLM's contribution from the schedule parameterization itself.

Agent scaffold differences. Frontier models (Claude, Gemini) use rich CLI agents; open-source models use structured-output prompting with injected context. While we ensure equivalent information access, the agent frameworks differ in their ability to iteratively debug failing schedules.

6 Conclusion

FunWake is a low-cost, open-source benchmark for LLM-discovered optimizer schedules. The task is simple—write a 4-output Python function—and evaluation is fast (30–60 seconds on CPU). Two findings justify the benchmark's design: (1) LLM agents discover genuinely novel schedule structures (decaying sinusoidal perturbation, dual Gaussian bumps) that generalize across farm sizes the agent never trained on, and (2) physics-based held-out evaluation is essential, as training AEP improvements of +9 GWh can correspond to held-out degradation.

The benchmark is not saturated: the gap between the best schedule (Gemini, +81.3 GWh on held-out) and the worst successful schedule (Qwen, +4.7 GWh) spans an order of magnitude, and the discovered

schedules differ qualitatively. We release all 700+ generated schedules, the evaluation harness, and seven farm variants at `https://github.com/kilojoules/funwake`.

References

Ying Chen, Hua Li, Kai Jin, and Qing Song. Wind farm layout optimization using genetic algorithm with different hub height turbines. *Energy Conversion and Management*, 70:56–65, 2013.

CO-Bench Authors. Co-bench: Benchmarking llm agents for combinatorial optimization. 2025. AAAI 2026.

FLLM Authors. Augmented-gda: Llm-augmented gradient descent algorithms. 2025. FLLM 2025.

Google DeepMind. Alphaevolve: A coding agent for scientific and algorithmic discovery. 2025. arXiv:2506.13131.

IEA Wind Systems. Iea wind 740-10-mw reference offshore wind plants. `https://github.com/IEAWindSystems/IEA-Wind-740-10-ROWP`, 2024.

Fei Liu, Xialiang Tong, Mingxuan Yuan, et al. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. 2024. arXiv:2401.02051.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. 2017.

G Mosetti, C Poloni, and B Diviacco. Optimization of wind turbine positioning in large windfarms by means of a genetic algorithm. *Journal of Wind Engineering and Industrial Aerodynamics*, 51:105–116, 1994.

Mads M Pedersen and Gunner C Larsen. Topfarm: Multi-fidelity optimization of wind farms. *Wind Energy Science*, 2019.

Julian Quick, Pierre-Elouan Rethore, and Mads M Pedersen. Speeding up large-wind-farm layout optimization using gradients, parallelization, and a heuristic algorithm for the initial layout. *Wind Energy Science*, 9: 321–341, 2024.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

Kevin Sim, Quentin Renau, and Emma Hart. Beyond the hype: Benchmarking llm-evolved heuristics for bin packing. In *EvoApplications*, 2025.

Jared Thomas, Andrew Baker, and Andrew Ning. Improving the accuracy of wind farm layout optimization with gradient-based methods. *Wind Energy Science*, 2023.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, et al. Large language models as optimizers. 2024.

Zhicheng Yao et al. Heurigym: An agentic benchmark for llm-crafted heuristics in combinatorial optimization. 2025. arXiv:2506.07972.

Haoran Ye, Jiarui Wang, Zhiguang Cao, and Guojie Song. Reevo: Large language models as hyper-heuristics with reflective evolution. In *NeurIPS*, 2024.